

Flag : `picoCTF{sm4ll_d_57ff60e1}`

📄 Résumé de la faille

Un service RSA génère la clé d'une façon inversée par rapport à la normale : au lieu de choisir `e` (petit, fixe) puis calculer `d`, le script choisit `d en premier, et petit` (256 bits, alors que `N` fait ~2096 bits), puis calcule `e` à partir de `d`.

```
d = getPrime(256)
e = inverse(d, phi)
```

Un `d` aussi petit par rapport à `N` rend RSA cassable via l'**attaque de Boneh-Durfee** (1999), qui permet de retrouver `d` à partir de `(N, e)` seuls, dès que `d < N0.292` environ.

🧠 Pourquoi c'est cassable : Boneh-Durfee en une phrase

RSA repose sur la difficulté de factoriser `N`. Boneh-Durfee ne factorise pas `N` directement — elle exploite une relation mathématique entre `e`, `d` et `N` (via `e*d ≡ 1 mod φ(N)`) grâce à une technique de **réduction de réseau (lattice reduction, LLL)**. Si `d` est petit, cette relation devient "resolvable" par cette technique, ce qui casse tout RSA construit ainsi, quelle que soit la taille de `N`.

🔧 Méthodo — les étapes

- 1. Lire le hint : "This might be a job for Boneh-Durfee" → attaque nommée explicitement, pas de doute sur la piste à suivre.
 - 2. Confirmer la faille en lisant le script fourni - `d = getPrime(256)` avant `e = inverse(d, phi)` → génération inversée, `d` anormalement petit → signature classique d'un challenge Boneh-Durfee.
 - 3. Chercher un outil pour exécuter l'attaque (pas la recoder à la main)
- Boneh-Durfee est une attaque mathématiquement complexe (construction de réseaux, polynômes multivariés, LLL) — en CTF, on utilise toujours une implémentation existante, jamais réécrite à la main.
- 4. Tentative RsaCtfTool (installé avec succès)

```
git clone https://github.com/Ganapati/RsaCtfTool
cd RsaCtfTool
pip install -r requirements.txt --break-system-packages
source venv/bin/activate
python3 -m RsaCtfTool -n N -e E --decrypt C --attack boneh_durfee --private
```

→ Erreur : `Can't load boneh_durfee because sage binary is not installed`. RsaCtfTool délègue cette attaque à un binaire **SageMath** externe qu'il ne fournit pas lui-même.

5. Tentative d'installation de SageMath en local (échec)

```
sudo apt install sagemath
```

→ Le paquet principal `sagemath` était absent/cassé dans les dépôts Kali au moment du challenge (seules ses bases de données annexes `sagemath-database-*` étaient disponibles). Une tentative d'ajouter le dépôt Debian bookworm a révélé un conflit de version Python insoluble (Sage 9.5 exige Python 3.10-3.12, incompatible avec la version installée sur Kali).

- 6. Solution retenue : SageMath en ligne, sans rien installer localement - CoCalc (cocalc.com) : environnement Sage complet dans le navigateur, gratuit, aucune installation. - Récupérer le script Boneh-Durfee de référence, utilisé dans la quasi-totalité des writeups CTF : github.com/mimoo/RSA-and-LLL-attacks/blob/master/boneh_durfee.sage - Créer un fichier `.sage` dans CoCalc, coller le script, remplacer les valeurs d'exemple `N` et `e` par les vraies valeurs du challenge. - Exécuter dans le terminal CoCalc :

```
sage solve.sage
```

→ Résultat en ~2 secondes :

```
private key found: 69930066959563146203434015254921533148746292485871654774717229563554026716709
```

7. Déchiffrement final : retour à du Python simple (pas besoin de Sage)

Une fois `d` récupéré, le déchiffrement RSA est une opération triviale (`m = cd mod n`) qui ne nécessite aucune bibliothèque de réseau — inutile de rester dans Sage pour cette étape.

```
from Crypto.Util.number import long_to_bytes

n = <N du challenge>
c = < ciphertext du challenge>
d = 69930066959563146203434015254921533148746292485871654774717229563554026716709

m = pow(c, d, n)
print(long_to_bytes(m))
```

→ `picoCTF{sm4ll_d_57ff60e1}` 🏆

Outil	Résultat
RsaCtfTool	Installé, mais dépend de Sage en interne pour cette attaque précise
<code>apt install sagemath</code>	Échec (paquet cassé / conflit Python sur Kali)
CoCalc (cocalc.com)	✔ Sage fonctionnel en ligne, script exécuté avec succès
Script mimoo/RSA-and-LLL-attacks	✔ Implémentation Boneh-Durfee de référence en CTF
Python + pycryptodome (Kali)	✔ Déchiffrement final une fois <code>d</code> connu

✔ Réflexes à retenir

- `d` **anormalement petit face à** `N` = vulnérabilité Boneh-Durfee (ou Wiener pour des cas encore plus extrêmes, `d < N^0.25`).
- Les attaques crypto avancées (Boneh-Durfee, Coppersmith...) ont presque toujours une **implémentation de référence déjà écrite** (souvent en SageMath) — chercher plutôt que recoder à la main.
- **Quand un outil système (apt) est cassé ou en conflit de versions, chercher une alternative en ligne** (CoCalc, SageMathCell) plutôt que de s'acharner à réparer l'installation locale — gain de temps énorme en CTF.
- **Ne pas tout faire dans le même environnement par principe** : Sage était nécessaire pour la partie mathématique complexe (LLL), mais le déchiffrement final (`pow(c, d, n)`) est un calcul simple qui se fait très bien en Python pur, sans y rester par automatisme.
- En Sage/Python, une variable définie **à l'intérieur d'une fonction** n'est pas accessible en dehors — pour tester rapidement, il est plus sûr d'ajouter du code directement dans la fonction existante (au bon niveau d'indentation) plutôt que de le sortir mal placé en bas du fichier.

🔧 Dans la vraie vie

Boneh-Durfee (1999) et Wiener (1990, son prédécesseur) sont des attaques historiques et bien documentées qui ont poussé les standards RSA à **imposer des tailles minimales pour** `d` (généralement, `d` doit être du même ordre de grandeur que `N` , jamais volontairement réduit pour "optimiser les performances" — un choix qui semble raisonnable en apparence mais détruit complètement la sécurité).